



Compiladores

Aula 10: Análise LL(1)

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Gramática transformada
- Decisão tabular
- AST
- Recuperação de erros

Gramática transformada

- LL(1):
 - $L \rightarrow$ Left-to-right:
lê a entrada da esquerda para a direita
 - $L \rightarrow$ Leftmost:
produz uma derivação mais à esquerda
 - $1 \rightarrow$ um lookahead:
decide com o próximo token

- Em cada passo, o parser escolhe uma única produção.

Gramática transformada

- A gramática de expressões

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow (E) \mid \text{id}$

E / E'

organizam a soma

T / T'

organizam a multiplicação

F

reconhece agrupamento ou id

Essa transformação preserva precedência e remove a recursão à esquerda.

Gramática transformada

- FIRST e FOLLOW:
 - FIRST inicia;
 - FOLLOW encerra e sincroniza.

FIRST(α)

O que pode iniciar uma cadeia derivada de α ?

```
FIRST(F) = { (, id }
FIRST(E') = { +, ε }
```

FOLLOW(A)

O que pode aparecer imediatamente depois de A?

```
FOLLOW(E) = { ), $ }
FOLLOW(T) = { +, ), $ }
```

Gramática transformada

- Quando a produção ϵ é escolhida?

$$E' \rightarrow + T E' \mid \epsilon$$

lookahead = +

aplica $E' \rightarrow + T E'$

lookahead $\in \text{FOLLOW}(E')$

aplica $E' \rightarrow \epsilon$

outro token

erro ou recuperação

Decisão tabular

- Construindo a tabela $M[A,a]$
- Para cada produção $A \rightarrow \alpha$:
 - $\text{FIRST}(\alpha)$:
insira $A \rightarrow \alpha$ nas colunas de $\text{FIRST}(\alpha)$
 - $\epsilon \in \text{FIRST}(\alpha)$:
também use as colunas de $\text{FOLLOW}(A)$
 - células vazias:
representam situações de erro

Uma gramática LL(1) possui no máximo uma produção em cada célula.

Decisão tabular

- A tabela da gramática de expressões

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Decisão tabular

- Parser preditivo: três componentes



Inicialização: pilha = [\$, E] • entrada termina em \$.

Decisão tabular

- Laço de análise com pilha explícita

$X \leftarrow$ topo da pilha;

$a \leftarrow$ token corrente

se $X = a = \$$ $\rightarrow$ aceitar

se X é terminal $\rightarrow$ casar e avançar

se $M[X,a] = \text{erro}$ $\rightarrow$ recuperar

senão $\rightarrow$ aplicar $X \rightarrow Y_1 \dots Y_n$

- A operação central:
 - desempilha X e empilha $Y_n \dots Y_2 Y_1$
 - Y_1 fica no topo.

Decisão tabular

- Simulação: $id + id * id$

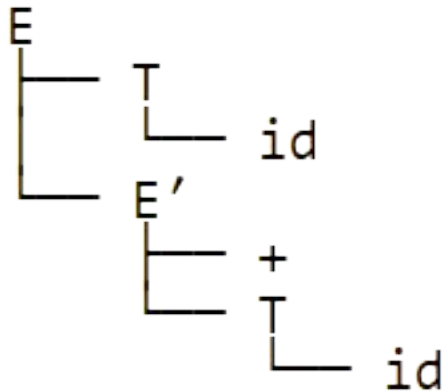
Pilha	Entrada	Ação
E \$	id + id * id \$	$M[E, id] = E \rightarrow TE'$
T E' \$	id + id * id \$	$M[T, id] = T \rightarrow FT'$
F T' E' \$	id + id * id \$	$M[F, id] = F \rightarrow id$
id T' E' \$	id + id * id \$	casa id
T' E' \$	+ id * id \$	$M[T', +] = T' \rightarrow \epsilon$
E' \$	+ id * id \$	$M[E', +] = E' \rightarrow +TE'$
+ T E' \$	+ id * id \$	casa +
T E' \$	id * id \$	$M[T, id] = T \rightarrow FT'$
F T' E' \$	id * id \$	$M[F, id] = F \rightarrow id$
id T' E' \$	id * id \$	casa id

A simulação continua até o casamento de \$ com \$.

AST

- As regras de produção são convertidas em nós da AST.

$E \rightarrow T E'$



produção

organiza a estrutura

token casado

fornece lexema e posição

ϵ

normalmente não gera nó

O parser reconhece e constrói: não são etapas isoladas.

Recuperação de erros

- Conflitos: quando uma célula recebe duas produções

$A \rightarrow \alpha \mid \beta$ e $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) \neq \emptyset$

Consequência: o lookahead não distingue as alternativas

Como corrigir?

- eliminar recursão à esquerda
- aplicar fatoração à esquerda
- recalcular FIRST e FOLLOW
- considerar LL(k), LR ou estratégia específica

Recuperação de erros

- Recuperação de erros por modo pânico
- O parser não precisa parar no primeiro erro: ele precisa encontrar um ponto seguro para continuar.
- Detectar:
 - terminal incompatível ou $M[A,a]$ vazia
- Descartar:
 - avançar tokens enquanto não houver sincronização
- Retomar:
 - usar FOLLOW(A), ;, },), ou início de comando

Mensagem útil: token encontrado + contexto esperado + linha e coluna.